



TUGAS BESAR

Teori Bahasa Formal dan Otomata



13524007 Muhammad Ashkar
13524039 Rhenaldy Cahyadi Putra
13524069 Miguel Rangga Deardo Sinaga
13524097 Muhammad Faiz Alfada Dharma



DAFTAR ISI

BAB I LANDASAN TEORI.....	4
1.1 Decorated Abstract Syntax Tree.....	4
1.2 Intermediate Code.....	4
1.3 Stack Machine.....	5
1.4 Interpreter.....	6
1.5 Instruction Set.....	6
BAB II PERANCANGAN DAN IMPLEMENTASI.....	8
2.1 Perancangan Program.....	8
2.1.1 Arsitektur Sistem.....	8
2.1.2 Teknologi yang Digunakan.....	8
2.1.3 Struktur Program.....	8
2.1.4 Fungsi atau Kelas Utama.....	9
2.1.5 Alur Kerja Program.....	9
2.1.6 Justifikasi Pilihan Implementasi.....	10
2.2 Implementasi Intermediate Code Generator.....	10
2.2.1 Inisialisasi Memori (INT).....	10
2.2.2 Translasi Literal dan Variabel.....	11
2.2.3 Translasi Operasi Aritmatika.....	11
2.2.4 Translasi Operasi Relasional.....	12
2.2.5 Translasi Statement Assignment.....	12
2.2.6 Translasi IF-ELSE.....	12
2.2.7 Translasi WHILE.....	13
2.2.8 Translasi WRITE dan WRITELN.....	13
2.3 Implementasi Interpreter.....	14
2.3.1 Representasi Memori dan Stack.....	14
2.3.2 Eksekusi Instruksi.....	14
2.3.3 Mekanisme Instruction Pointer.....	15
2.3.4 Eksekusi Operasi OPR.....	15
2.4 Implementasi Error Handling.....	16
2.4.1 Invalid Instruction.....	16
2.4.2 Invalid Jump Target.....	17
2.4.3 Stack Error.....	17
2.4.4 Arithmetic Error.....	18
2.4.5 Error Lain yang Diimplementasikan.....	19
2.5 Hasil Implementasi.....	19
2.5.1 IntermediateCodeGenerator.....	19
2.5.2 StackInterpreter.....	20
2.5.3 Main.....	21



BAB III PENGUJIAN.....	23
3.1 Test Case 1.....	23
3.2 Test Case 2.....	23
3.3 Test Case 3.....	24
3.4 Test Case 4.....	26
3.5 Test Case 5.....	27
BAB IV DIAGRAM.....	29
4.1 Class Diagram.....	29
4.2 Flowchart Program.....	32
BAB V KESIMPULAN & SARAN.....	33
5.1 Kesimpulan.....	33
5.2 Saran.....	33
LAMPIRAN.....	34
REFERENSI.....	34



BAB I LANDASAN TEORI

1.1 Decorated Abstract Syntax Tree

Decorated Abstract Syntax Tree (Decorated AST) merupakan bentuk lanjutan dari AST yang telah dilengkapi dengan informasi semantik hasil tahap semantic analysis. Decorated AST tidak hanya merepresentasikan struktur logis program, tetapi juga menyimpan informasi semantik seperti tipe data setiap ekspresi (integer, boolean, string, dan lain-lain), referensi ke simbol pada symbol table, dan informasi scope atau cakupan variabel.

Informasi ini sangat penting dalam proses kompilasi lanjutan karena compiler tidak lagi hanya memahami struktur program, tetapi juga makna dari setiap elemen di dalamnya. Dengan Decorated AST, proses translasi ke bentuk yang lebih rendah (seperti intermediate code) dapat dilakukan secara tepat, karena setiap node sudah memiliki konteks semantik yang jelas. Sebagai contoh, ekspresi

```
x := a + b
```

pada AST biasa hanya menunjukkan operasi penjumlahan, tetapi pada Decorated AST, compiler sudah mengetahui apakah a dan b bertipe integer atau string sehingga dapat menentukan instruksi yang sesuai, apakah operasi aritmatika atau operasi seperti konkatenasi string.

Pada tahap ini, Decorated AST berperan sebagai input utama bagi Intermediate Code Generator. Seluruh validasi semantik telah diselesaikan pada tahap sebelumnya, sehingga generator hanya berfokus pada proses translasi. Informasi seperti tipe data, referensi simbol, scope, dan alamat variabel digunakan untuk menentukan instruksi intermediate code yang tepat, termasuk pemetaan identifier ke alamat memori agar operasi load dan store dapat dieksekusi dengan benar oleh interpreter.

1.2 Intermediate Code

Intermediate Code (kode antara) adalah representasi program yang berada di antara level AST dan kode mesin virtual. Tujuan utama intermediate code adalah untuk menyederhanakan struktur program yang awalnya berbentuk pohon (tree) menjadi bentuk linier berupa daftar instruksi.

Pada tugas ini, intermediate code yang digunakan berbentuk Three Address Code (TAC) atau instruksi stack-based sederhana. Setiap instruksi hanya melakukan satu operasi



dasar seperti memuat nilai (LIT, LOD), menyimpan nilai (STO), perubahan alur kontrol (JMP, JPC), dan lain sebagainya. Sebagai contoh, ekspresi

```
x := 5 + 2
```

dapat diterjemahkan menjadi Intermediate Code sebagai berikut:

```
LIT 0 5  
LIT 0 2  
OPR 0 2  
STO 0 3
```

Instruksi tersebut secara berurutan memuat nilai 5 dan 2 ke dalam stack, melakukan operasi penjumlahan, kemudian menyimpan hasilnya ke alamat memori variabel x. Melalui pendekatan ini, struktur program yang kompleks dapat direpresentasikan sebagai urutan instruksi linier yang lebih mudah dieksekusi oleh interpreter.

Dapat disimpulkan bahwa intermediate code memiliki peran krusial sebagai jembatan antara analisis program (front-end) dan eksekusi program (back-end). Dengan adanya intermediate code, interpreter tidak perlu lagi memahami struktur AST yang kompleks, tetapi cukup menjalankan instruksi sederhana secara berurutan.

1.3 Stack Machine

Stack Machine merupakan model mesin eksekusi yang menggunakan struktur data stack sebagai media utama penyimpanan operand dan hasil operasi dengan prinsip LIFO (Last In First Out). Berbeda dengan arsitektur berbasis register, seluruh operasi pada Stack Machine dilakukan melalui mekanisme penambahan (push) dan pengambilan (pop) nilai dari stack.

Dalam arsitektur ini, setiap instruksi intermediate code akan memanipulasi stack secara langsung. Instruksi seperti LIT dan LOD akan mendorong nilai ke stack, sedangkan instruksi STO akan mengambil nilai dari stack untuk disimpan ke memori. Sementara itu, instruksi OPR digunakan untuk melakukan operasi aritmatika atau logika dengan mengambil operand dari stack, kemudian menyimpan kembali hasilnya ke stack. Sebagai contoh, berikut eksekusi stack untuk ekspresi:

```
x := 5 + 2
```

dengan Intermediate Code:

```
LIT 0 5
```



```
LIT 0 2
OPR 0 2
STO 0 3
```

Perubahan isi stack selama eksekusi dapat dilihat pada tabel berikut:

Langkah	Instruksi	Isi Stack
1	LIT 0 5	[5]
2	LIT 0 2	[5, 2]
3	OPR 0 2 (ADD)	[7]
4	STO 0 x	[]

Melalui mekanisme ini, Stack Machine menyederhanakan evaluasi ekspresi menjadi operasi berbasis struktur data sederhana, sehingga tidak memerlukan register kompleks seperti pada arsitektur mesin nyata.

1.4 Interpreter

Interpreter merupakan komponen yang bertugas mengeksekusi Intermediate Code yang telah dihasilkan oleh Intermediate Code Generator. Interpreter membaca instruksi secara berurutan, menjalankan operasi yang sesuai, serta memperbarui kondisi stack dan memori selama proses eksekusi berlangsung.

Pada milestone ini, interpreter bekerja layaknya sebuah virtual machine (VM) yang mengeksekusi instruksi menggunakan Instruction Pointer (IP) dengan siklus fetch (mengambil instruksi), decode (menganalisis instruksi), dan execute (menjalankan instruksi). Setiap instruksi yang dieksekusi akan memengaruhi state stack maupun memori program, seperti operasi pemuatan nilai, penyimpanan variabel, serta evaluasi ekspresi aritmatika dan logika.

1.5 Instruction Set



Instruction set adalah kumpulan instruksi dasar yang dapat dipahami dan dieksekusi oleh interpreter. Secara umum, instruction set dapat dibagi menjadi beberapa kategori, yaitu instruksi pemuatan nilai, penyimpanan data, operasi komputasi, serta kontrol alur eksekusi. Berikut adalah beberapa instruksi utama yang digunakan dalam implementasi:

Instruksi	Nama Operasi	Deskripsi
LIT	Load Literal	Memasukkan nilai konstanta ke stack
LOD	Load	Mengambil nilai dari alamat memori ke stack
STO	Store	Menyimpan nilai teratas stack ke memori
OPR	Operation	Menjalankan operasi aritmatika/logika

Selain instruksi utama yang telah dijelaskan pada tabel, terdapat juga instruksi lain seperti JMP, JPC, dan RET yang digunakan untuk mendukung kontrol alur eksekusi program serta proses terminasi. Instruksi-instruksi tersebut melengkapi instruction set sehingga interpreter dapat menangani alur program secara keseluruhan, termasuk percabangan, perulangan, dan penghentian eksekusi.



BAB II PERANCANGAN DAN IMPLEMENTASI

2.1 Perancangan Program

2.1.1 Arsitektur Sistem

Program Arion Interpreter dirancang dengan arsitektur bertahap yang mengikuti alur kerja interpreter. Tahapan awal dimulai dari proses lexical analysis, syntax analysis, dan semantic analysis. Pada milestone ini, program dirancang untuk difokuskan pada tahap mengubah hasil AST yang sudah valid menjadi intermediate code, kemudian menjalankan intermediate code tersebut menggunakan interpreter berbasis stack.

2.1.2 Teknologi yang Digunakan

Implementasi Milestone 4 dikembangkan menggunakan bahasa pemrograman C++ sebagai kelanjutan dari milestone-milestone sebelumnya. Kode Milestone 4 ditempatkan pada folder `src/interpreter/` dan dipisahkan ke beberapa file agar lebih modular dan mudah dikembangkan.

Program memanfaatkan Standard Template Library (STL) C++, khususnya `std::vector` untuk menyimpan daftar instruksi intermediate code dan stack memori eksekusi, serta `std::map` untuk pemetaan nama variabel ke alamat memori dan penyimpanan nilai konstanta. `std::unique_ptr` digunakan untuk manajemen kepemilikan memori secara otomatis pada node-node AST yang diterima sebagai input. Selain itu, `std::ostringstream` digunakan untuk mengakumulasi output program sebelum dikembalikan sebagai string.

Fitur C++ modern yang dimanfaatkan antara lain `dynamic_cast` untuk pengecekan dan konversi jenis node AST secara aman saat traversal, serta `enum class` untuk mendefinisikan OpCode instruksi dengan type-safety. Tidak ada library eksternal di luar STL yang dibutuhkan, sehingga proses kompilasi tetap sederhana menggunakan Makefile yang sama dengan milestone sebelumnya.

2.1.3 Struktur Program

```
JKW-Tubes-IF2224-2026/
├── src/
│   ├── main.cpp
│   ├── lexer/
│   ├── parser/
│   ├── semantic/
│   ├── utils/
│   │   └── exception.hpp
│   └── interpreter/
│       ├── opcode.hpp
│       ├── opcode.cpp
│       ├── value.hpp
│       └── instruction.hpp
```




```
├── instruction.cpp
├── irgenerator.hpp
├── irgenerator.cpp
├── stackinterpreter.hpp
├── stackinterpreter.cpp
├── test/
│   └── milestone-4/
│       ├── test1.txt
│       ├── test2.txt
│       ├── test3.txt
│       ├── test4.txt
│       └── test5.txt
```

2.1.4 Fungsi atau Kelas Utama

1. opcode.hpp
Mendefinisikan enum class OpCode yang merepresentasikan seluruh instruksi stack-machine: LIT, LOD, STO, CAL, INT_, JMP, JPC, OPR, dan RET. Fungsi opcodeName() mengonversi enum ke string untuk keperluan pencetakan.
2. value.hpp
Struct Value merepresentasikan nilai runtime yang dapat bertipe integer, real, boolean, string, atau char. Struct ini memiliki factory method statis (Value::integer(), Value::real(), Value::boolean(), dst.) serta method konversi (asInteger(), asReal(), asBoolean(), toOutputString()).
3. instruction.hpp – Struct Instruction
Struct Instruction menyimpan satu baris Intermediate Code, terdiri dari field op (OpCode), level (level leksikal), operand (integer), serta literal dan hasLiteral untuk instruksi LIT dengan nilai non-integer. Fungsi instructionToString() dan instructionsToString() digunakan untuk mencetak daftar instruksi
4. irgenerator.hpp
Kelas utama yang melakukan dua tugas, yakni Generasi Intermediate Code untuk traversal Decorated AST yang menghasilkan vektor instruksi. Selain itu, Interpreter/Eksekusi untuk eksekusi vektor instruksi menggunakan Stack Machine.
5. stackInterpreter.hpp
Kelas StackInterpreter didefinisikan pada file stackinterpreter.hpp dan diimplementasikan pada stackinterpreter.cpp. Kelas ini bertugas menjalankan daftar instruksi intermediate code menggunakan model stack machine.

2.1.5 Alur Kerja Program

Alur kerja keseluruhan program setelah Milestone 4 ditambahkan:

1. Input: main.cpp membaca path file kode sumber Arion.
2. Lexical Analysis (Milestone 1): Lexer menghasilkan daftar token.



3. Syntax Analysis (Milestone 2): Parser membangun Parse Tree.
4. AST Construction (Milestone 3): ASTBuilder mengonversi Parse Tree menjadi AST.
5. Semantic Analysis (Milestone 3): ASTVisitor mendekorasi AST dan mengisi Symbol Table.
6. Intermediate Code Generation (Milestone 4): IntermediateCodeGenerator melakukan traversal Decorated AST dan menghasilkan vektor instruksi.
7. Interpretation (Milestone 4): Stack Machine mengeksekusi vektor instruksi dan mencetak output.

2.1.6 Justifikasi Pilihan Implementasi

Rancangan implementasi ini dibuat agar selaras dengan arsitektur stack machine. Representasi opcode mengeliminasi kebutuhan alokasi register virtual yang kompleks dalam kode antara. Penanganan operasi evaluasi ekspresi dan pemanggilan fungsi dapat direduksi menjadi serangkaian operasi tumpukan primitif, yang secara signifikan menyederhanakan logika internal pemrosesan kode perantara sekaligus mengoptimalkan efisiensi performa eksekusi interpreter.

Pada aspek pengelolaan memori internal modul generator, penggunaan struktur data `std::map<std::string, int>` untuk `variableAddress` dipilih untuk efisiensi pemetaan pengenalan variabel ke alamat memori relatif. Aturan inisialisasi penanda alamat variabel ditentukan secara konstan dimulai dari indeks 3 (`nextAddress = 3`). Keputusan perancangan ini diambil karena indeks 0, 1, dan 2 dialokasikan secara eksklusif sebagai linkage area lingkungan eksekusi blok (berturut-turut untuk penyimpanan static link, dynamic link, dan return address). Skema pemisahan ruang ini menjamin validitas manajemen pemanggilan fungsi bersarang dan mencegah terjadinya tumpang tindih (memory-overwrite) antar-lingkup variabel.

2.2 Implementasi Intermediate Code Generator

Intermediate Code Generator merupakan komponen yang bertugas mengubah AST hasil parsing dan semantic analysis menjadi daftar instruksi intermediate code. Kelas utama yang digunakan adalah `IntermediateCodeGenerator`. Kelas ini menyimpan daftar instruksi dalam atribut `code`, menyimpan pemetaan variabel ke alamat memori melalui `variableAddress`, menyimpan konstanta pada `constants`, serta menggunakan `nextAddress` untuk menentukan alamat memori variabel berikutnya.

Instruksi intermediate code direpresentasikan menggunakan `Instruction` yang didefinisikan pada `instruction.hpp`, sedangkan jenis opcode didefinisikan melalui `OpCode` pada `opcode.hpp`. Instruksi yang tersedia adalah LIT, LOD, STO, CAL, INT, JMP, JPC, OPR, dan RET.

2.2.1 Inisialisasi Memori (INT)



Generator terlebih dahulu membaca deklarasi variabel dan konstanta melalui fungsi `collectDeclaration()`. Setiap variabel diberi alamat memori mulai dari indeks 3 karena index 0, 1, dan 2 disiapkan untuk static link, dynamic link, dan return address.

Contoh:

$A \rightarrow 3$

$B \rightarrow 4$

$C \rightarrow 5$

Setelah semua variabel dikumpulkan, generator membuat instruksi INT untuk menentukan ukuran memori awal.

2.2.2 Translasi Literal dan Variabel

- LIT

Literal diterjemahkan menggunakan instruksi LIT. Literal yang didukung adalah integer, real, boolean, string, dan char. Nilai literal disimpan menggunakan struct Value. Contoh :

$S \rightarrow \text{LIT } 0 \ S$

$\text{'Hello'} \rightarrow \text{LIT } 0 \ \text{'Hello'}$

$\text{True} \rightarrow \text{LIT } 0 \ \text{true}$

- LOD

Variabel diterjemahkan menggunakan LOD untuk mengambil nilai dari alamat memori. Contoh, jika variabel A berada di alamat 3, maka:

$A \rightarrow \text{LOD } 0 \ 3$

- STO

Instruksi STO digunakan untuk menyimpan nilai ke alamat variabel. Contoh:

$\text{STO } 0 \ 3$

2.2.3 Translasi Operasi Aritmatika

Operasi aritmatika diterjemahkan menjadi instruksi OPR dengan nomor operasi tertentu. Translasi operasi dilakukan oleh fungsi `operationCodeFor()`. Operasi aritmatika yang didukung adalah:

- ADD : + $\rightarrow$ OPR 0 2
- SUB : - $\rightarrow$ OPR 0 3
- MUL : * $\rightarrow$ OPR 0 4
- DIV : / $\rightarrow$ OPR 0 5
- DIV : div $\rightarrow$ OPR 0 5
- MOD : mod $\rightarrow$ OPR 0 6



Generator memproses ekspresi biner dengan pola post-order. Artinya, operand kiri dibangkitkan terlebih dahulu, kemudian operand kanan, lalu operatornya.

2.2.4 Translasi Operasi Relasional

Operasi relasional digunakan untuk menghasilkan nilai boolean pada kondisi percabangan dan perulangan. Sama seperti operasi aritmatika, operasi relasional juga diterjemahkan menjadi instruksi OPR. Operasi relasional yang didukung adalah:

- EQL : $= \rightarrow$ OPR 0 7
- NEQ : $<> \rightarrow$ OPR 0 8
- LSS : $< \rightarrow$ OPR 0 9
- GEQ : $>= \rightarrow$ OPR 0 10
- GTR : $> \rightarrow$ OPR 0 11
- LEQ : $<= \rightarrow$ OPR 0 12

2.2.5 Translasi Statement Assignment

Statement assignment diterjemahkan dengan cara membangkitkan instruksi untuk ekspresi di sisi kanan terlebih dahulu. Setelah ekspresi selesai diproses, hasilnya berada pada stack. Generator kemudian menghasilkan instruksi STO untuk menyimpan hasil tersebut ke alamat variabel tujuan. Contoh:

akan diterjemahkan menjadi,

A := 5
LIT 0 5
STO 3

2.2.6 Translasi IF-ELSE

Statement if-else diterjemahkan menggunakan instruksi JPC dan JMP. Instruksi JPC digunakan untuk melompat apabila kondisi bernilai false, sedangkan instruksi JMP digunakan untuk melewati blok else setelah blok then selesai dijalankan.

Karena alamat else dan alamat akhir belum diketahui ketika instruksi jump dibuat, generator menggunakan fungsi patchOperand(). Fungsi ini memperbarui operand instruksi JPC atau JMP setelah alamat tujuan sebenarnya diketahui. Contoh:

```
if x > 5 then
    y := x * 2
else
    y := x - 1;
```

Secara umum akan diterjemahkan menjadi,

LOD 0 3



```
LIT 0 5
OPR 0 11
JPC 0 alamat_else
LOD 0 3
LIT 0 2
OPR 0 4
STO 0 4
JMP 0 alamat_akhir
LOD 0 3
LIT 0 1
OPR 0 3
STO 0 4
```

Jika statement if tidak memiliki blok else, maka JPC langsung diarahkan ke instruksi setelah blok then.

2.2.7 Translasi WHILE

Statement while diterjemahkan dengan menyimpan alamat awal kondisi. Setelah kondisi diproses, generator membuat instruksi JPC untuk keluar dari loop jika kondisi bernilai false. Setelah body loop selesai, generator menghasilkan instruksi JMP untuk kembali ke alamat awal kondisi. Contoh:

```
while i < 5 do
  begin
    i := i + 1;
  end;
```

Diterjemahkan menjadi bentuk umum,

```
LOD 0 alamat_i
LIT 0 5
OPR 0 9
JPC 0 alamat_akhir
LOD 0 alamat_i
LIT 0 1
OPR 0 2
STO 0 alamat_i
JMP 0 alamat_awal
```

Selama kondisi bernilai true, body loop akan terus dijalankan. Ketika kondisi bernilai false, instruksi JPC akan mengarahkan eksekusi ke instruksi setelah loop.

2.2.8 Translasi WRITE dan WRITELN



Procedure bawaan `write` dan `writeln` diterjemahkan menjadi instruksi OPR. Pada implementasi saat ini, hanya kedua procedure tersebut yang didukung secara khusus oleh generator. Pemetaan operasinya adalah,

`write` → OPR 0 13
`writeln` → OPR 0 14

Jika `write` atau `writeln` memiliki beberapa argumen, generator akan memproses argumen satu per satu. Pada `writeln`, newline hanya diberikan pada argumen terakhir. Jika `write` atau `writeln` tidak memiliki argumen, generator akan menghasilkan literal string kosong.

2.3 Implementasi Interpreter

Interpreter pada implementasi terbaru direalisasikan melalui kelas `StackInterpreter` yang berada pada file `stackinterpreter.hpp` dan `stackinterpreter.cpp`. Kelas ini bertugas mengeksekusi daftar instruksi intermediate code yang dihasilkan oleh `IntermediateCodeGenerator`.

Interpreter bekerja menggunakan model stack machine. Setiap instruksi dibaca berdasarkan nilai instruction pointer, kemudian dijalankan sesuai opcode. Hasil eksekusi disimpan pada stack dan output program dikumpulkan menggunakan `ostringstream`.

2.3.1 Representasi Memori dan Stack

Memori dan stack direpresentasikan menggunakan `vector<Value>` bernama `stack`. Struct `Value` digunakan untuk menyimpan nilai runtime dengan tipe integer, real, boolean, string, char, atau unknown.

Selain stack, interpreter juga memiliki atribut output untuk menyimpan hasil dari `write` dan `writeln`, serta atribut `ip` sebagai instruction pointer.

Pada awal eksekusi, fungsi `run()` akan mengosongkan stack, mengosongkan output, dan mengatur `ip` ke 0. Instruksi `INT` kemudian digunakan untuk menentukan ukuran awal stack.

2.3.2 Eksekusi Instruksi

Eksekusi instruksi dilakukan pada fungsi `run()`. Interpreter membaca instruksi berdasarkan nilai `ip`, lalu menjalankan aksi sesuai opcode.

Instruksi yang dijalankan adalah,

- `INT` → untuk menyiapkan ukuran memori awal
- `LIT` → untuk menyimpan literal pada instruksi
- `LOD` → untuk memuat nilai dari alamat memori



- STO → untuk menyimpan nilai ke alamat memori
- JMP → untuk lompatan tanpa syarat
- JPC → untuk lompatan bersyarat apabila kondisi bernilai false
- OPR → untuk operasi aritmatika, relasional, boolean, dan output
- RET → sebagai penanda akhir program

Pada instruksi INT, interpreter melakukan resize terhadap stack sesuai operand instruksi. Pada instruksi LIT, nilai literal dimasukkan ke stack menggunakan fungsi `push()`. Pada instruksi LOD, interpreter mengambil nilai dari alamat memori menggunakan fungsi `load()`, lalu memasukkannya ke stack. Pada instruksi STO, interpreter mengambil nilai teratas stack menggunakan `pop()`, lalu menyimpannya ke alamat tertentu menggunakan `store()`.

Instruksi JMP mengubah nilai ip ke alamat tujuan. Instruksi JPC mengambil nilai kondisi dari stack; jika kondisi bernilai false, ip diarahkan ke operand instruksi, sedangkan jika true, eksekusi lanjut ke instruksi berikutnya. Instruksi OPR menjalankan operasi berdasarkan nomor operasi. Instruksi RET menghentikan eksekusi dan mengembalikan string output program.

2.3.3 Mekanisme Instruction Pointer

Instruction pointer atau ip digunakan untuk menunjukkan instruksi yang sedang dijalankan. Pada awal eksekusi, ip bernilai 0. Untuk instruksi biasa seperti LIT, LOD, STO, dan OPR, nilai ip akan bertambah satu setelah instruksi selesai dijalankan. Contoh:

```
0 LIT 0 5
1 LIT 0 10
2 OPR 0 2
```

Interpreter akan menjalankan instruksi indeks 0, kemudian 1, kemudian 2. Untuk instruksi JMP dan JPC, nilai ip tidak selalu bertambah satu. Instruksi JMP akan langsung mengubah ip ke alamat tujuan. Pada JPC, interpreter mengambil kondisi dari stack terlebih dahulu. Jika kondisi bernilai false, ip diubah ke target jump. Jika kondisi bernilai true, ip hanya dinaikkan satu.

Sebelum melakukan lompatan, interpreter memvalidasi target jump menggunakan fungsi `validateInstructionPointer()` agar tidak melompat ke alamat instruksi yang tidak tersedia.

2.3.4 Eksekusi Operasi OPR

Instruksi OPR digunakan untuk menjalankan operasi aritmatika, relasional, boolean, dan output. Eksekusi operasi dilakukan oleh fungsi `executeOperation()`. Daftar operasi yang digunakan adalah:

```
1 → NEG
```



- 2 → ADD
- 3 → SUB
- 4 → MUL
- 5 → DIV
- 6 → MOD
- 7 → EQL
- 8 → NEQ
- 9 → LSS
- 10 → GEQ
- 11 → GTR
- 12 → LEQ
- 13 → WRT
- 14 → WRTLN
- 15 → AND
- 16 → OR
- 17 → NOT

Untuk operasi aritmatika seperti ADD, SUB, MUL, DIV, dan MOD, interpreter mengambil dua nilai teratas dari stack. Nilai yang pertama diambil menjadi operand kanan, sedangkan nilai berikutnya menjadi operand kiri. Hasil operasi kemudian dimasukkan kembali ke stack.

Untuk operasi perbandingan seperti EQL, NEQ, LSS, GEQ, GTR, dan LEQ, interpreter menghasilkan nilai boolean. Nilai boolean ini dapat digunakan oleh instruksi JPC.

Untuk operasi output, OPR 0 13 digunakan sebagai write, sedangkan OPR 0 14 digunakan sebagai writeln. Perbedaannya adalah writeln menambahkan newline setelah mencetak nilai.

2.4 Implementasi Error Handling

Error handling digunakan untuk mencegah generator menghasilkan intermediate code dari AST yang tidak valid atau dari fitur yang belum didukung. Pada implementasi ini, error pada tahap intermediate code generation dilempar menggunakan `InterpreterGenerateError`, sedangkan error pada tahap runtime interpreter dilempar menggunakan `InterpreterRuntimeError`.

2.4.1 Invalid Instruction

Invalid instruction dapat terjadi ketika generator menemukan ekspresi, statement, atau operator yang belum didukung. Pada fungsi `operationCodeFor()`, generator hanya menerima operator yang sudah dipetakan ke nomor operasi OPR. Jika operator tidak dikenali, generator akan menghasilkan error:

Operator belum didukung: <operator>



Jika generator menemukan jenis ekspresi yang belum didukung, maka error yang dihasilkan adalah

Jenis ekspresi belum didukung

Jika generator menemukan statement yang belum didukung, maka error yang dihasilkan adalah:

Jenis statement belum didukung.

Pada tahap interpreter, invalid instruction juga ditangani ketika nomor operasi OPR tidak dikenali. Jika nomor operasi tidak tersedia, interpreter menghasilkan error:

OPR tidak dikenal: <nomor_operasi>

2.4.2 Invalid Jump Target

Invalid jump target berkaitan dengan instruksi control flow seperti JMP dan JPC. Pada tahap generator, target jump ditangani menggunakan mekanisme patching melalui fungsi patchOperand(). Fungsi patchOperand() menerima indeks instruksi yang ingin diperbarui dan operand baru sebagai alamat tujuan. Sebelum memperbarui operand, fungsi ini memeriksa apakah indeks instruksi valid. Jika indeks patch berada di luar ukuran vector code, generator menghasilkan error:

patch target intermediate code tidak valid

Pada tahap runtime, StackInterpreter memvalidasi target jump menggunakan fungsi validateInstructionPointer(). Jika instruksi JMP atau JPC mencoba melompat ke alamat di luar daftar intermediate code, interpreter menghasilkan error:

InvalidJumpTarget: target <target> di luar intermediate code

2.4.3 Stack Error

Stack error ditangani langsung oleh StackInterpreter. Fungsi push() memeriksa apakah ukuran stack sudah melebihi batas maksimum maxStackSize. Jika batas terlampaui, interpreter menghasilkan error:

StackOverflow: stack melebihi batas interpreter

Fungsi pop() memeriksa apakah stack kosong sebelum mengambil nilai. Jika stack kosong, interpreter menghasilkan error:

StackUnderflow: pop pada stack kosong

Selain itu, akses memori pada instruksi LOD dan STO divalidasi melalui fungsi load() dan store(). Jika alamat memori lebih kecil dari 0 atau lebih besar atau sama dengan ukuran stack, interpreter menghasilkan error:



`IndexOutOfBoundsException`: address <alamat> tidak ada di stack

Pada tahap generator, error terkait alamat variabel juga dicegah melalui fungsi `addressOf()`. Jika variabel tidak ditemukan pada `variableAddress`, generator menghasilkan error:

alamat variabel tidak ditemukan: <nama_variabel>

2.4.4 Arithmetic Error

Arithmetic error ditangani pada operasi numerik di `StackInterpreter`. Fungsi `numericBinary()` memastikan operand yang digunakan valid secara numerik. Jika operasi aritmatika selain penjumlahan dilakukan terhadap operand non-numerik, interpreter menghasilkan error:

operasi aritmatika membutuhkan operand numerik

Untuk operasi pembagian, interpreter memeriksa apakah pembagi bernilai nol. Jika pembagi bernilai nol, interpreter menghasilkan error:

division by zero

Untuk operasi modulo, interpreter juga memeriksa pembagi. Jika pembagi bernilai nol, interpreter menghasilkan error:

modulo by zero

Interpreter juga memeriksa overflow dan underflow integer 32-bit melalui fungsi `ensureInt32()`. Jika hasil operasi melebihi batas maksimum integer 32-bit, interpreter menghasilkan error:

`OverflowError`: hasil integer melebihi batas 32-bit

Jika hasil operasi lebih kecil dari batas minimum integer 32-bit, interpreter menghasilkan error:

`UnderflowError`: hasil integer di bawah batas 32-bit

Selain itu, `struct Value` juga memiliki validasi konversi. Jika nilai diminta sebagai numerik tetapi tipenya tidak sesuai, error yang dihasilkan adalah:

value bukan numerik

Jika nilai diminta sebagai integer tetapi tidak dapat dikonversi, error yang dihasilkan adalah:

value bukan integer



2.4.5 Error Lain yang Diimplementasikan

Terdapat beberapa error lain yang sudah ditangani pada tahap generator. Pertama, root AST harus berupa ProgramNode. Pada fungsi generate(), root AST dicek menggunakan `dynamic_cast<ProgramNode*>`. Jika root bukan program, generator menghasilkan error:

root AST bukan ProgramNode

Kedua, konstanta harus dapat dievaluasi ketika proses generate intermediate code. Jika bentuk konstanta tidak dapat diterjemahkan menjadi Value, generator menghasilkan error:

konstanta tidak dapat dievaluasi saat generate intermediate code

Ketiga, ekspresi kosong tidak diperbolehkan. Jika fungsi generateExpression() menerima node kosong, generator menghasilkan error:

ekspresi kosong

Keempat, literal angka divalidasi saat diproses. Jika literal integer atau real gagal dikonversi, generator menghasilkan error:

literal angka tidak valid: <nilai>

Kelima, pemanggilan fungsi pada ekspresi belum didukung. Jika generator menemukan ProcCallNode sebagai ekspresi, generator menghasilkan error:

terjadi kesalahan pada pemanggilan fungsi: <nama_fungsi>

Keenam, procedure atau function call selain write dan writeln belum didukung. Jika ditemukan pemanggilan selain dua procedure tersebut, generator menghasilkan error:

terjadi kesalahan saat pemanggilan fungsi/prosedur: <nama_procedure>

Ketujuh, jika instruksi INT memiliki ukuran negatif, interpreter menghasilkan error:

ukuran INT negatif

2.5 Hasil Implementasi

2.5.1 IntermediateCodeGenerator



```
std::vector<Instruction> IntermediateCodeGenerator::generate(ASTNode* root) {
    code.clear();
    variableAddress.clear();
    constants.clear();
    nextAddress = 3;

    auto* program = dynamic_cast<ProgramNode*>(root);
    if (!program) {
        throw InterpreterGenerateError("root AST bukan ProgramNode");
    }

    for (auto& decl : program->declarations) {
        collectDeclaration(decl.get());
    }

    emit(Instruction(OpCode::INT_, 0, nextAddress));
    generateStatement(program->mainBlock.get());
    emit(Instruction(OpCode::RET, 0, 0));
    return code;
}

const std::map<std::string, int>& IntermediateCodeGenerator::getVariableAddressMap() const {
    return variableAddress;
}
```

Potongan kode tersebut adalah bagian utama dari IntermediateCodeGenerator. Fungsi tersebut akan menerima AST yang sudah dibuat pada tahap sebelumnya, dan akan diproses menjadi serangkaian instruksi yang akan diproses pada tahap berikutnya.

2.5.2 StackInterpreter



```
std::string StackInterpreter::run() {
    stack.clear();
    output.str("");
    output.clear();
    ip = 0;

    while (ip >= 0 && ip < static_cast<int>(code.size())){
        const Instruction& instruction = code[ip];
        switch (instruction.op){
            case OpCode::INT_:
                if (instruction.operand < 0) throw InterpreterRuntimeError("ukuran INT negatif");
                stack.resize(static_cast<size_t>(instruction.operand), Value::integer(0));
                ++ip;
                break;
            case OpCode::LIT:
                push(instruction.literal);
                ++ip;
                break;
            case OpCode::LOD:
                push(load(instruction.operand));
                ++ip;
                break;
            case OpCode::STO: {
                Value value = pop();
                store(instruction.operand, value);
                ++ip;
                break;
            }
            case OpCode::JMP:
                validateInstructionPointer(instruction.operand);
                ip = instruction.operand;
                break;
            case OpCode::JPC: {
                Value condition = pop();
                if (!condition.asBoolean()){
                    validateInstructionPointer(instruction.operand);
                    ip = instruction.operand;
                }
                else{
                    ++ip;
                }
                break;
            }
            case OpCode::OPR:
                executeOperation(instruction.operand);
                ++ip;
                break;
            case OpCode::CAL:
```

Potongan kode tersebut adalah bagian utama dari kode interpreter. Kode tersebut akan menginisialisasi variabel-variabel yang akan digunakan untuk menyimpan hasil dari interpreter. Input dari fungsi tersebut adalah rangkaian instruksi yang akan dijalankan untuk menghasilkan output. Lalu fungsi akan melakukan loop untuk memproses semua instruksi yang ada dan akan mengembalikan hasil dari instruksi yang diproses.

2.5.3 Main



```
// Milestone 4
IntermediateCodeGenerator generator;
std::vector<Instruction> code = generator.generate(astRoot.get());
StackInterpreter interpreter(code);
std::string programOutput = interpreter.run();

std::ostringstream result;
result << "==== INSTRUCTION ====\\n";
result << instructionsToString(code);
result << "\\n==== PROGRAM OUTPUT ====\\n";
result << programOutput;

cout << result.str();

ofstream output4("test/milestone-4/output.txt");
output4 << result.str() << endl;
output4.close();
```

Pada milestone ini, ditambahkan kode berikut pada bagian main.cpp untuk memanggil dan menampilkan output dari interpreter. Program pertama akan memanggil fungsi generate untuk mengubah AST menjadi rangkaian instruksi, lalu rangkaian instruksi tersebut akan diproses oleh StackInterpreter sehingga menghasilkan output dari kode.



BAB III PENGUJIAN

3.1 Test Case 1

Masukkan (Input)
<pre>program Arithmetic; var a, b, c: integer; begin a := 5; b := a + 10; c := (a + b) * 2; writeln('c=', c) end.</pre>
Keluaran (output)
<pre>===== INSTRUCTION ===== 0 INT 0 6 1 LIT 0 5 2 STO 0 3 3 LOD 0 3 4 LIT 0 10 5 OPR 0 2 6 STO 0 4 7 LOD 0 3 8 LOD 0 4 9 OPR 0 2 10 LIT 0 2 11 OPR 0 4 12 STO 0 5 13 LIT 0 'c=' 14 OPR 0 13 15 LOD 0 5 16 OPR 0 14 17 RET ===== PROGRAM OUTPUT ===== c=40</pre>

3.2 Test Case 2

Masukkan (Input)



```
program IfElseTest;
var
  x, y: integer;
begin
  x := 10;
  if x > 5 then
    y := x * 2
  else
    y := x - 1;
  writeln(y)
end.
```

Keluaran (output)

```
===== INSTRUCTION =====
0 INT 0 5
1 LIT 0 10
2 STO 0 3
3 LOD 0 3
4 LIT 0 5
5 OPR 0 11
6 JPC 0 12
7 LOD 0 3
8 LIT 0 2
9 OPR 0 4
10 STO 0 4
11 JMP 0 16
12 LOD 0 3
13 LIT 0 1
14 OPR 0 3
15 STO 0 4
16 LOD 0 4
17 OPR 0 14
18 RET

===== PROGRAM OUTPUT =====
20
```

3.3 Test Case 3

Masukkan (Input)

```
program TestConstEnumRange;

const
  MAX == 100;
  MIN == -10;
```




```
PI == 3.14;
MSG == 'hello';
CH == 'a';

type
  Direction == (north, south, east, west);
  Month == (jan, feb, mar, apr, may, jun);
  Letter == 'a'..'z';
  UpperLetter == 'A'..'Z';

var
  d: integer;
  i: integer;

begin
  d := 5;
  i := MAX;

  case d of
    0: writeln('nol');
    1, 2, 3: writeln('kecil');
    4, 5: writeln('sedang');
    6, 7, 8, 9: writeln('besar')
  end;

  case i of
    100: writeln('maksimum')
  end;
end.
```

Keluaran (output)

```
===== INSTRUCTION =====
0 INT 0 5
1 LIT 0 1
2 STO 0 3
3 LIT 0 0
4 STO 0 4
5 LOD 0 3
6 LIT 0 5
7 OPR 0 12
8 JPC 0 18
9 LOD 0 4
10 LOD 0 3
11 OPR 0 2
12 STO 0 4
13 LOD 0 3
14 LIT 0 1
15 OPR 0 2
16 STO 0 3
17 JMP 0 5
```



```
18 LOD 0 4
19 OPR 0 14
20 RET
```

```
===== PROGRAM OUTPUT =====
15
```

3.4 Test Case 4

Masukkan (Input)

```
program M4RepeatFor;
var
  i, result: integer;
begin
  result := 0;
  for i := 1 to 3 do
  begin
    result := result + i
  end;
  repeat
    result := result - 1
  until result == 3;
  writeln(result)
end.
```

Keluaran (output)

```
===== INSTRUCTION =====
0 INT 0 5
1 LIT 0 0
2 STO 0 4
3 LIT 0 1
4 STO 0 3
5 LOD 0 3
6 LIT 0 3
7 OPR 0 12
8 JPC 0 18
9 LOD 0 4
10 LOD 0 3
11 OPR 0 2
12 STO 0 4
13 LOD 0 3
14 LIT 0 1
```



```
15 OPR 0 2
16 STO 0 3
17 JMP 0 5
18 LOD 0 4
19 LIT 0 1
20 OPR 0 3
21 STO 0 4
22 LOD 0 4
23 LIT 0 3
24 OPR 0 7
25 JPC 0 18
26 LOD 0 4
27 OPR 0 14
28 RET
```

```
===== PROGRAM OUTPUT =====
3
```

3.5 Test Case 5

Masukkan (Input)

```
program M4ConstBoolean;

const
  LIMIT == 4;
  MSG == 'ok';

var
  i: integer;

begin
  i := LIMIT;
  if (i == 4) and not (i < 0) then
    writeln(MSG)
  else
    writeln('fail')
  end.
end.
```

Keluaran (output)

```
===== INSTRUCTION =====
0 INT 0 4
1 LIT 0 4
2 STO 0 3
3 LOD 0 3
```



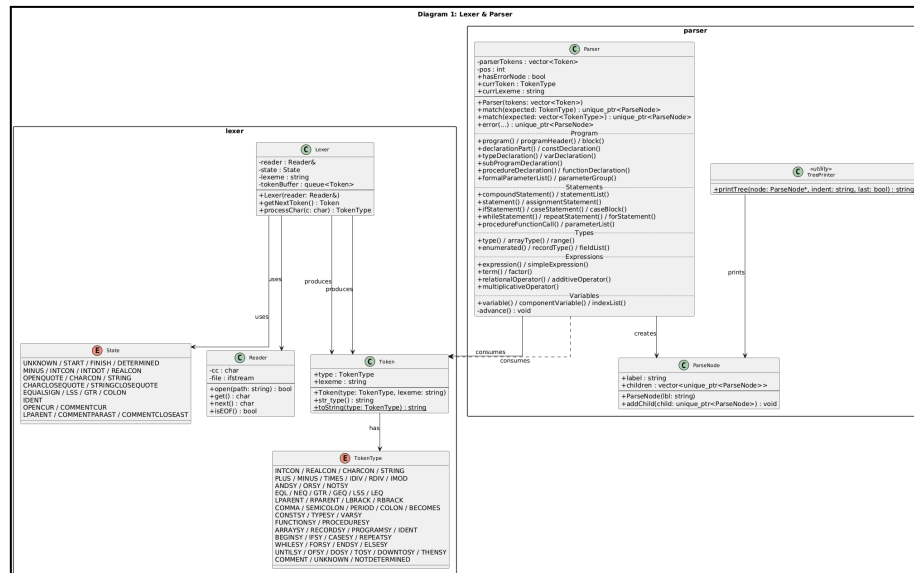
```
4 LIT 0 4
5 OPR 0 7
6 LOD 0 3
7 LIT 0 0
8 OPR 0 9
9 OPR 0 17
10 OPR 0 15
11 JPC 0 15
12 LIT 0 'ok'
13 OPR 0 14
14 JMP 0 17
15 LIT 0 'fail'
16 OPR 0 14
17 RET
```

```
===== PROGRAM OUTPUT =====
ok
```



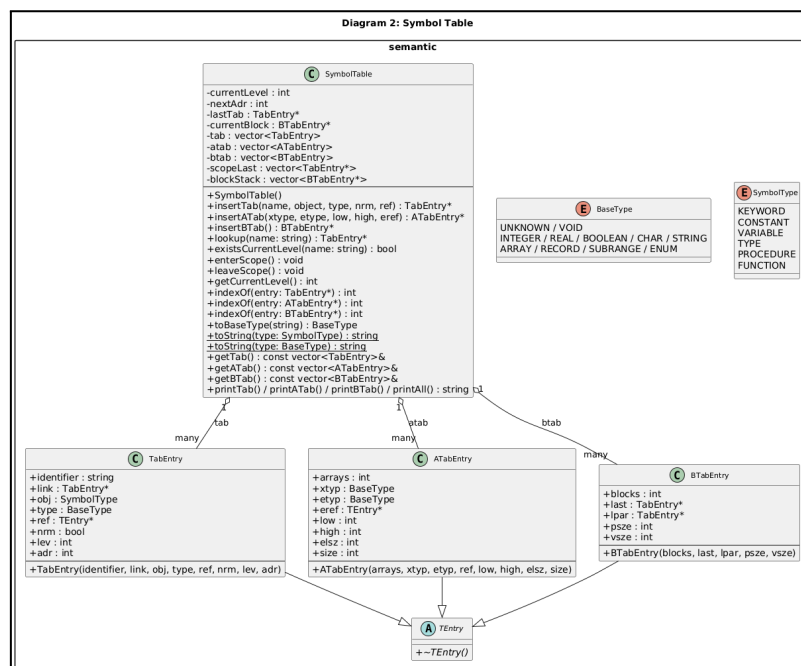
BAB IV DIAGRAM

4.1 Class Diagram



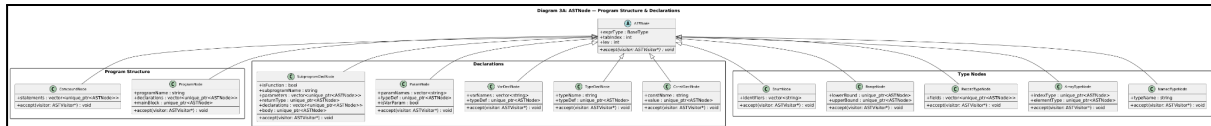
Gambar 4.1.1 – Diagram Kelas Lexer & Parser

Diagram ini menggambarkan komponen analisis leksikal dan sintaksis, mencakup kelas Reader, Lexer, Token, serta enumerasi TokenType dan State, beserta kelas Parser dan ParseNode.



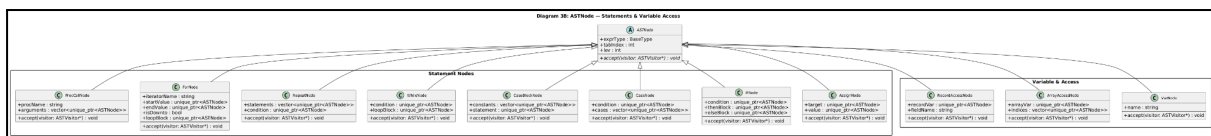
Gambar 4.1.2 – Diagram Kelas Symbol Table

Diagram ini menggambarkan struktur tabel simbol yang digunakan pada tahap analisis semantik, mencakup hierarki TEntry beserta turunannya (TabEntry, ATabEntry, BTabEntry) dan kelas SymbolTable yang mengelola scope, level leksikal, serta penyimpanan informasi identifer.



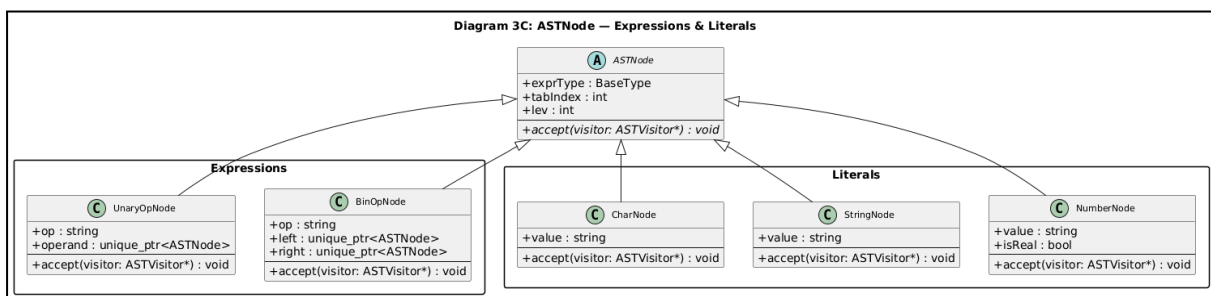
Gambar 4.1.3 – Diagram Kelas AST Node: Program Structure, Declarations & Types

Diagram ini menggambarkan node-node Abstract Syntax Tree yang merepresentasikan struktur program, deklarasi konstanta, tipe, variabel, parameter, dan subprogram, serta definisi tipe array, record, range, dan enum sebagai turunan dari kelas abstrak ASTNode.



Gambar 4.1.4 – Diagram Kelas AST Node: Statements & Variable Access

Diagram ini menggambarkan node-node Abstract Syntax Tree yang merepresentasikan pernyataan seperti assignment, if, case, while, repeat, for, dan pemanggilan prosedur, serta akses variabel sederhana, elemen array, dan field record sebagai turunan dari kelas abstrak ASTNode.



Gambar 4.1.5 – Diagram Kelas AST Node: Expressions & Literals

Diagram ini menggambarkan node-node Abstract Syntax Tree yang merepresentasikan ekspresi operasi biner dan unary, serta literal bilangan, string, dan karakter sebagai turunan dari kelas abstrak ASTNode.

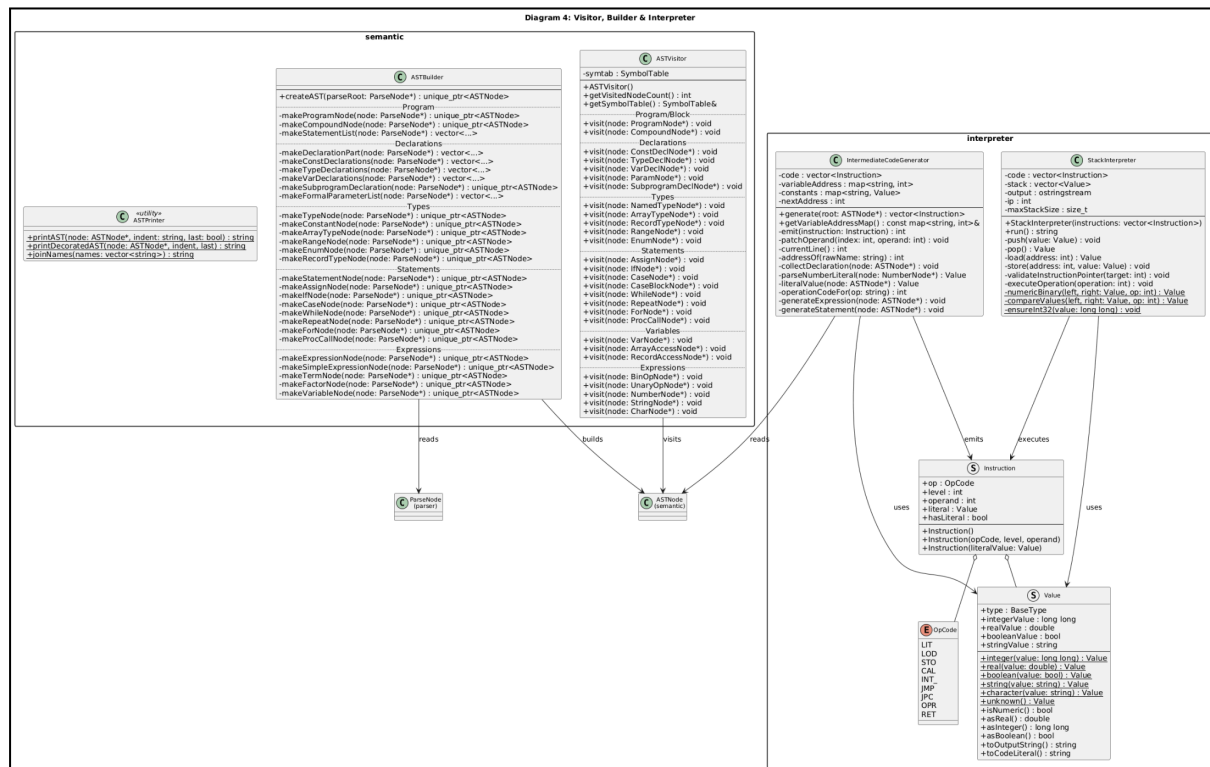


Diagram ini menggambarkan komponen pemrosesan lanjutan, meliputi ASTVisitor untuk analisis semantik, ASTBuilder untuk konversi ParseNode menjadi ASTNode, IntermediateCodeGenerator untuk pembangkitan bytecode, serta StackInterpreter yang mengeksekusi instruksi menggunakan Value, Instruction, dan OpCode.

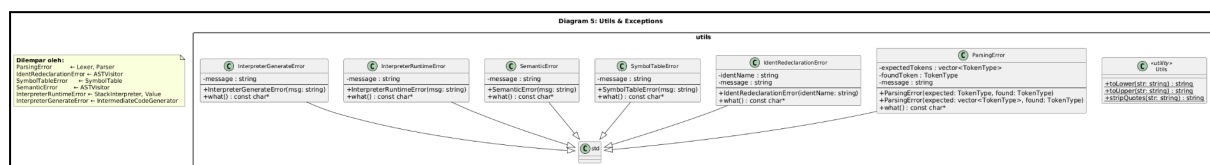
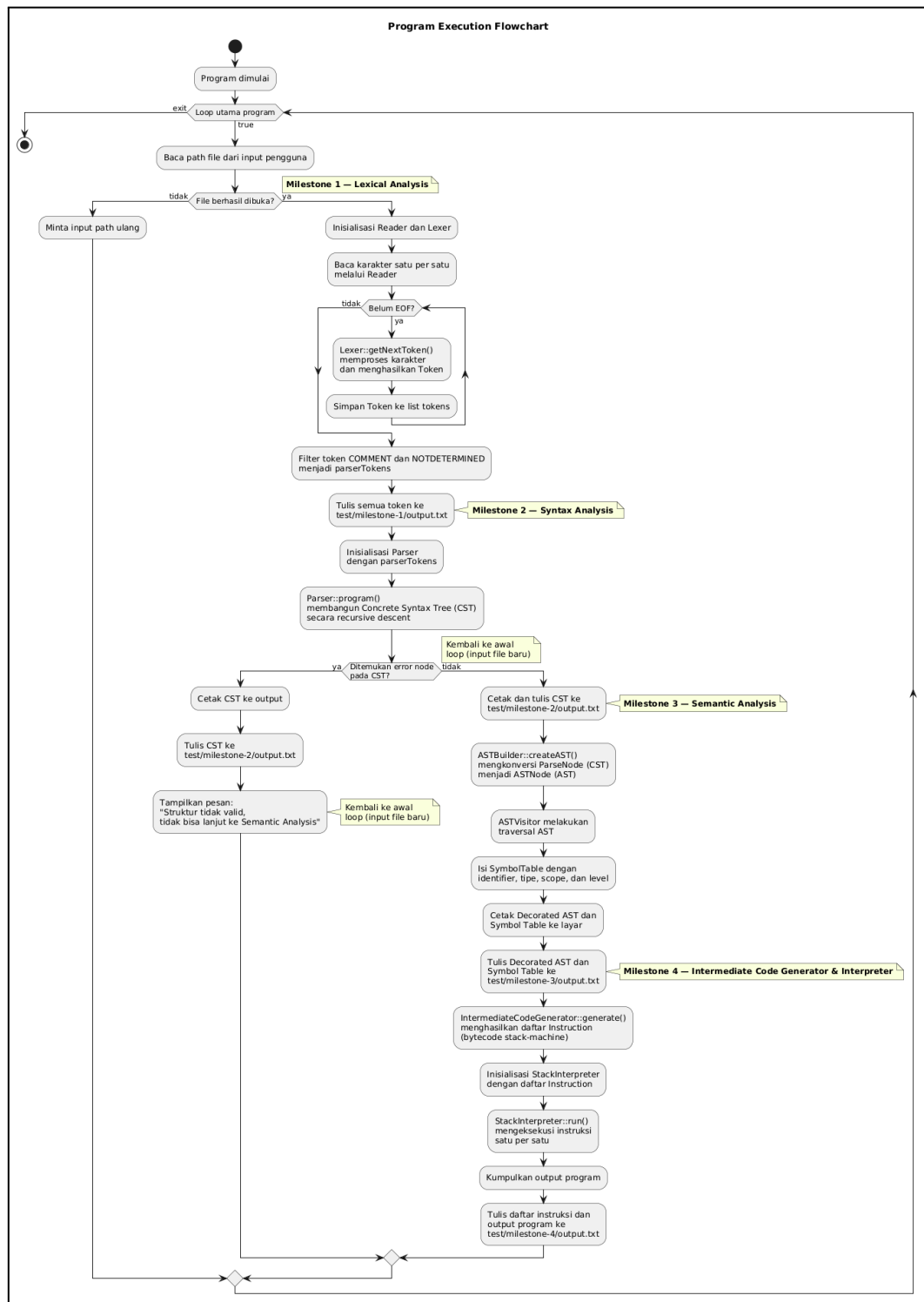


Diagram ini menggambarkan kelas-kelas pendukung yang digunakan di seluruh pipeline kompilasi, mencakup utilitas string pada kelas Utils serta hierarki exception meliputi `ParsingError`, `SemanticError`, `SymbolTableError`, `IdentRedeclarationError`, `InterpreterRuntimeError`, dan `InterpreterGenerateError`.



4.2 Flowchart Program





BAB V KESIMPULAN & SARAN

5.1 Kesimpulan

Berdasarkan implementasi milestone ini, dapat disimpulkan bahwa proses penerjemahan program dari Decorated Abstract Syntax Tree (AST) hingga eksekusi menggunakan Interpreter telah berhasil direalisasikan melalui beberapa tahapan utama, yaitu pembangkitan Intermediate Code, eksekusi berbasis Stack Machine, serta pengelolaan instruksi oleh Interpreter. Setiap komponen memiliki peran penting dalam membentuk alur kompilasi dan eksekusi program secara keseluruhan, sehingga program dapat dijalankan sesuai dengan spesifikasi bahasa Arion.

5.2 Saran

Pengembangan lebih lanjut dapat difokuskan pada perluasan instruction set agar lebih lengkap dan mendukung fitur bahasa tingkat lanjut, serta peningkatan mekanisme penanganan error agar sistem lebih aman dalam menghadapi kondisi runtime yang tidak valid. Dari sisi implementasi, perbaikan dokumentasi dan modularitas kode juga dapat dilakukan untuk mempermudah pengembangan dan pemeliharaan sistem pada tahap selanjutnya.



LAMPIRAN

Link Repository Github

<https://github.com/Faizalfadaa/JKW-Tubes-IF2224-2026>

Pembagian Tugas

NIM	Kontribusi
13524007	25%
13524039	25%
13524069	25%
13524097	25%

REFERENSI

"Compilers - Principles, Techniques, and Tools (2006)"

[https://repository.unikom.ac.id/48769/1/Compilers%20-%20Principles,%20Techniques,%20and%20Tools%20\(2006\).pdf](https://repository.unikom.ac.id/48769/1/Compilers%20-%20Principles,%20Techniques,%20and%20Tools%20(2006).pdf)